***The Handshake Problem***

***– Space Complexity (Detailed Analysis)***

***Program***:

```
// Handshake Problem Recursion

// This code calculates the number of handshakes that occur
when a given number of people are in a room. It uses
recursion to calculate the number of handshakes based on the
number of people.
#include <iostream>
using namespace std;

int handshake(int n) // Function to calculate the number of
handshakes
{
    if (n == 0 || n == 1) // Base case
    {
        return 0; // If there is only one person in the
room, there are no handshakes
    }
    cout << n << endl;
    return handshake(n - 1) + (n - 1); // Recursive call :
n-1 people shake hands with each other and the remaining
person shakes hands with n-1 people
}

int main() // Main function
{
    int n;
 // Number of people
    cout << "Enter the number of people: ";
  // Input the number of people
    cin >> n;
  // Store the number of people
    cout << "The number of handshakes are: " <<
handshake(n); // Output the number of handshakes
```

```
    return 0;
  // Return statement
}
```

$$\textit{Space Complexity}$$

$$A.\textit{Input Space Complexity}$$

# 1. Parametric Input Space

- $\textit{Only parameter is } n \ (\textit{an integer})$
- $\textit{Size of an integer} = \textit{constant} \ (e.g., 4 \textit{ bytes})$
- $\textit{No extra input structures} \ (\textit{arrays, lists, etc.})$

$$\textit{Parametric Input Space} = \Theta(1)$$

# 2. Full (Holistic) Input Space

$\textit{Now think deeper}$:

$$\textit{What is the real} - \textit{world input of the handshake problem}?$$

- $\textit{The problem says: } n \textit{ people in a room}$
- $\textit{If we explicitly represent them, we might store}$:

```
Person people[n];
```

$\textit{That would take}: O(n)$

$But\ according\ to\ the\ program\ ,it\ only\ have$:

```
    int n;
```

$So$:

- $Logical\ problem\ input\ (Holistic\ Input) \rightarrow\ \Theta(n)$
- $Program\ input\ representation(Parametric\ Input)\ \rightarrow\ \Theta(1)$

$Therefore\ ,based\ on\ program - level\ i.e.according\ to\ the\ code\ we\ will\ take,$

$Parametric\ Input\ as\ `final\ Input\ Space\ `\ i.e.\Theta(1).$

# $Auxiliary\ Space$

## $Includes$:

- $recursion\ stack$
- $local\ variables$
- $temporary\ data$

$Each\ recursive\ call\ creates\ a\ stack\ frame\ containing$:

- $parameter\ n$
- $return\ address$
- $temporary\ value\ (n-1)$
- $I/O\ context\ for\ cout$

$Each\ frame\ =\ \Theta(1) space\ per\ recursive\ call$

## ***Recursion Structure***

***Call sequence***:

$handshake(n) \longrightarrow \boxed{Max.Depth = n}$

$\downarrow$

$handshake(n-1) \longrightarrow \boxed{Depth = n-1}$

$\downarrow$

$handshake(n-2) \longrightarrow \boxed{Depth = n-2}$

$\downarrow$

$\ldots$

$handshake(1) \longrightarrow \boxed{Depth = 1}$

$$\boldsymbol{Hence\ ,we\ take\ here\ the\ Maximum\ Depth = n}$$

$$\therefore \boldsymbol{Total\ Auxiliary\ Space = Maximum\ Depth \times Space\ per\ Recursive\ Call.}$$

$$= \boldsymbol{\Theta(n) \times \Theta(1)}$$

$$= \boldsymbol{\Theta(n)}$$

## $\boldsymbol{Total\ Space\ Complexity}$:

$$\boldsymbol{Input\ Space + Auxiliary\ Space = \Theta(1) + \Theta(n)}$$
$$\boldsymbol{= \Theta(n)}.$$

---